Study Guide: Foundations of Generative AI - Training, Inference, Context, Tokenization, and RAG

This study guide provides an authoritative, source-grounded review of autoregressive language models based on *Foundations of Generative Artificial Intelligence* by Marco Cremaschi (November 2026). It analyzes how raw text is tokenized and numerically embedded, how models operate during pretraining and ordinary inference, how external context and retrieval-augmented generation (RAG) function, and why next-token probabilities must be strictly distinguished from factual truth.

1. Tokenization and Numerical Representations

Understanding an autoregressive language model begins with tracing the sequence of transformations that convert human-readable text into numerical vectors and contextualized states.

Tokenization Algorithms and Vocabulary Indices

As detailed in Chapter 1.3, raw text input does not enter a language model directly as words or characters. Instead, a tokenizer converts a string of text into a sequence of numerical identifiers known as token IDs.

Key principles of tokenization include:

- **Token Units**: A token is a variable-length vocabulary unit. Depending on the tokenizer, a single token can represent a whole word, part of a word, punctuation, or a character sequence that includes spaces (such as a preceding space marking word boundaries).
- **Vocabulary Indices**: Token IDs are integer indices pointing to entries in a predefined vocabulary dictionary. The specific integer assigned to a token (for instance, ID 785 or ID 15344) carries no scalar measurement of meaning or semantic distance. Token IDs that are numerically close to one another do not share similar meanings.
- **String Sensitivity**: Segmentation depends strictly on the tokenizer algorithm, character capitalisation, and exact spacing. For example, in Chapter 1.3, inspecting the Qwen tokenizer output for the string "The capital of Italy is" yields five distinct tokens: `The` (ID 785), ` capital` (ID 6722), ` of` (ID 315), ` Italy` (ID 15344), and ` is` (ID 374).
- **Context Length Measurement**: Context limits in language model architectures are measured exclusively in tokens, rather than word or character counts. Different languages, writing conventions, and tokenizers produce different token counts for equivalent passages.

From Embeddings to Contextual Representations

Once token IDs are generated, Chapter 1.4 maps their progression through three distinct stages of numerical representation:

1. **Initial Embedding**: Each token ID acts as a lookup key to retrieve an initial embedding vector from a learned weight matrix. This embedding represents the token in isolation before any contextual interaction occurs.
2. **Positional Encoding**: Because Transformer models process input sequences in parallel during training and lack inherent recurrent order, positional information must be added. Architectures incorporate positional encodings-such as Rotary Position Embedding (RoPE) described in Chapter 1.4 for Qwen2.5-to encode spatial position within the sequence.

3. **Contextual Representation**: As vectors pass through successive Transformer decoder blocks, causal attention mechanisms enable representations to interact across positions. Consequently, a token's representation is transformed dynamically based on surrounding tokens. The same token ID takes on different contextual representations when placed in different sentences.

2. Data, Parameters, and Pretraining vs. Ordinary Inference

A fundamental conceptual distinction exists between altering a model's internal parameters through training and running calculations with static parameters during ordinary inference.

Parameters, Pretraining, and Instruction Alignment

As documented in Chapter 1.2, parameters are the learned numerical weights that govern transformations inside the model. Before training, parameters do not contain an organized, readable encyclopedia of facts.

During the pretraining phase:

- The system processes large text corpora using an autoregressive next-token prediction objective.
- The model computes a predicted probability distribution over the vocabulary and compares it against actual target tokens in the text examples to calculate an aggregate prediction error.
- Optimization algorithms update the model's weight parameters to minimize this prediction error.
- Subsequent alignment stages, such as instruction tuning (for example, training with human preferences as documented for InstructGPT by Ouyang et al., 2022, cited in Chapter 1.2), adapt parameter behaviors to follow user instructions. However, instruction tuning does not guarantee factual truth or eliminate model bias.

Mechanics of Ordinary Inference

Chapter 1.2 and Chapter 1.5 define ordinary inference as performing forward-pass calculations using a fixed, unchanging set of pre-existing parameters.

During ordinary inference:

- **Fixed Model Weights**: The model's parameter values remain completely static. Changing the prompt or context alters the input values of the calculation, but does not modify the underlying model.
- **Transformer Decoder Flow**: As illustrated in Chapter 1.5, input vectors pass through decoder blocks consisting of causal self-attention, feed-forward sublayers, residual connections, and layer normalization.
- **Causal Attention Masking**: Causal attention enforces a strict sequence constraint: when predicting position i, the model can attend to current and preceding tokens ($\leq i$), but future token positions are masked out (Chapter 1.5).
- **Logit Production**: The final layer produces unnormalized numerical scores called **logits** for every token entry in the vocabulary dictionary.

3. Context Window Dynamics and Retrieval-Augmented Generation (RAG)

Context management allows applications to supply dynamic information to an inference call without modifying the underlying neural network parameters.

Context Window Mechanics

Chapter 1.2 and Chapter 1.7 state that context comprises the entire sequence of information supplied to the current inference pass. This sequence includes system instructions, user prompts, conversation history, and retrieved background documents.

When a user inserts a definition, custom rule, or document passage into a prompt, the newly provided text influences the forward pass of that specific inference run. However, as emphasized in Chapter 1.2 and Chapter 1.7, providing information in the context window **does not update model parameters**. When the inference pass ends, the model's weights remain identical to their state before the prompt was received.

Retrieval-Augmented Generation (RAG) Architecture

Retrieval-Augmented Generation (Lewis et al., 2020; Chapter 1.7) decouples external document collections from the fixed model parameters:

- **Retrieval Component**: An external search module queries an indexed document database to identify passages relevant to the user's prompt.
- **Prompt Augmentation**: The retrieved passages are formatted and prepended into the context window alongside the user's question.
- **Conditioned Generation**: The fixed language model performs ordinary inference, generating an answer conditioned on both the query and the retrieved text excerpts.

As highlighted in Chapter 1.1 and Chapter 1.7, RAG expands the available information in context without undergoing retraining. However, RAG performance depends on retrieval quality: if the retriever returns irrelevant text, or if the model misinterprets retrieved passages, the generated response may still contain inaccuracies.

4. Next-Token Probability vs. Factual Truth

A central theme across Chapter 1.6 and Chapter 1.8 is the strict distinction between statistical continuation probabilities and real-world factual truth.

Softmax Transformation and Temperature

At the end of a forward pass, the model outputs raw logits z_i for each vocabulary token. Chapter 1.6 details how the Softmax function normalizes these logits into non-negative probabilities p_i that sum to 1 across the full vocabulary:

$$p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

where $T > 0$ represents the temperature parameter.

Holding logits fixed, temperature adjusts the concentration of the output distribution:

- **Lower Temperature ($T < 1.0$)**: Concentrates probability mass onto high-scoring logits, making selection more deterministic.
- **Higher Temperature ($T > 1.0$)**: Flattens the distribution, allocating relatively more probability mass to lower-scoring vocabulary tokens.
- **Order Preservation**: Temperature scaling modifies probability values but leaves the ranking order of logits strictly unchanged.

To illustrate this mechanism, Chapter 1.6 presents a worked three-token example with fixed logits ($z_A = 2, z_B = 1, z_C = 0$):

- At $T = 1.0$, Token A receives 66.52%, Token B 24.47%, and Token C 9.00%.
- At $T = 0.5$, Token A concentrates to 86.68%, Token B drops to 11.73%, and Token C drops to 1.59%.
- At $T = 2.0$, Token A flattens to 50.65%, Token B shifts to 30.72%, and Token C rises to 18.63%.

Statistical Probability is Not Factual Veracity

As stressed in Chapter 1.6, Chapter 1.8, and Study Resource B:

- **Statistical Continuation Metric**: The probability p_i assigned to a token measures how statistically likely that token is to continue the prompt string based on patterns learned during pretraining. It does not measure the factual truth of the resulting sentence.
- **High Probability ≠ Truth**: A token with an 80% output probability means that under the current logit distribution, it has an 80% likelihood of selection. It does **not** signify an 80% probability of being factually true.
- **Hallucinations and Temperature**: Chapter 1.8 notes that model errors and hallucinations do not vanish when temperature is reduced or when greedy decoding is applied. Lowering temperature simply causes the model to reproduce its top-ranked prediction more deterministically-even if that prediction is factually false.

5. Comparative Analysis: Training vs. Ordinary Inference vs. RAG

To clarify how different interventions alter model states and outputs, the following table contrasts Training, Ordinary Inference, and Retrieval-Augmented Generation across their defining technical dimensions (synthesized from Chapter 1.2, Chapter 1.5, and Chapter 1.7).

Operational Dimension	Pretraining / Fine-Tuning	Ordinary Inference	Retrieval-Augmented Generation (RAG)
Primary Objective	Modify internal weight parameters to minimize aggregate prediction error on text examples (Chapter 1.2).	Execute forward-pass transformations over input tokens to compute output logits and probabilities (Chapter 1.5).	Retrieve relevant external document excerpts and condition response generation on retrieved text (Chapter 1.7).
Parameter State	**Updated**: Model weight values are permanently changed via optimization gradients (Chapter 1.2).	**Fixed**: Model parameters remain strictly constant throughout computation (Chapter 1.2).	**Fixed**: Model parameters remain unchanged; only context content is modified (Chapter 1.7).
Input Signal	Large-scale pretraining text corpora or instruction-preference demonstration pairs (Chapter 1.2).	User prompt, context history, and system instructions (Chapter 1.2).	User question plus retrieved external document passages inserted into context (Chapter 1.7).
Core Mechanism	Backpropagation, loss calculation, and optimization weight updates (Chapter 1.2).	Causal self-attention, feed-forward sublayers, and Softmax normalization (Chapter 1.5).	External document retrieval + context window insertion + fixed-model forward pass (Chapter 1.7).
Primary Output	An updated model parameter checkpoint file (Chapter 1.2).	Next-token probability distribution over the vocabulary and output text (Chapter 1.6).	Text response conditioned on external retrieved document passages (Chapter 1.7).
Knowledge Source	Internalized parametric memory embedded in learned weights during training (Chapter 1.2).	Statistical regularities in fixed weights applied to current context (Chapter 1.2).	External document collection indexed and queried dynamically per request (Chapter 1.7).

6. Short-Answer Assessment Questions

Test your knowledge of the core concepts with the following three short-answer questions.

Question 1: Tokenization and Representations

Explain why a token ID is distinct from both its initial embedding vector and its contextual representation. How does causal self-attention modify a token's representation during inference?

Question 2: Parameter States in Context Insertion and RAG

A researcher pastes a 30-page research paper into a prompt window or uses RAG to query an external library of articles. Does this interaction update the language model's parameters? Explain the operational difference between expanding contextual availability and fine-tuning parameters.

Question 3: Next-Token Probability vs. Factual Veracity

If an autoregressive model outputs a top candidate token with a 90% Softmax probability, does this guarantee a 90% probability that the generated statement is factually true? How does Softmax temperature affect distribution concentration, and why can low temperature settings never eliminate hallucinations?

7. Answer Key and Detailed Solutions

Answer 1 Solution

- **Chapter References**: Chapter 1.3, Chapter 1.4, Chapter 1.5.
- **Explanation**:
  1. A **token ID** is an integer index in a vocabulary table; it possesses no vector direction or semantic measurement (Chapter 1.3).
  2. An **initial embedding** is a static vector retrieved using the token ID from a learned weight table, representing the token in isolation (Chapter 1.4).
  3. A **contextual representation** is the dynamic vector produced after passing through Transformer decoder layers. Through **causal attention** (Chapter 1.5), representations at position i integrate information from permitted preceding tokens ($\leq i$), allowing the same token ID to yield distinct vector representations depending on sentence context.

Answer 2 Solution

- **Chapter References**: Chapter 1.2, Chapter 1.7, Study Resource B.
- **Explanation**:
  1. **No**, supplying text in context or querying documents via RAG does **not** update model parameters (Chapter 1.2, Chapter 1.7).
  2. Parameters are the internal weight values of the neural network, which are modified exclusively during pretraining or fine-tuning via gradient optimization.
  3. Context insertion and RAG make external passages available during the forward pass of a fixed model (Chapter 1.7). Once inference finishes, model parameters remain unchanged. RAG alters input context, not parameter weights.

Answer 3 Solution

- **Chapter References**: Chapter 1.6, Chapter 1.8, Study Resource B.
- **Explanation**:
  1. **No**, a 90% Softmax probability does **not** indicate a 90% probability of factual truth.
  2. Softmax probabilities measure statistical continuation likelihood over the vocabulary based on training patterns, not real-world factual accuracy (Chapter 1.6, Chapter 1.8).
  3. **Temperature (T)** scales logits prior to Softmax. Lowering temperature concentrates probability mass onto top-ranked logits, making output selection more deterministic, but it preserves logit ranking and adds zero factual knowledge (Chapter 1.6). If a model's learned logits favor an incorrect claim, setting a low temperature simply causes the model to output that hallucination deterministically with high statistical confidence (Chapter 1.8). Factual correctness must be verified against external primary sources.